data
fecha

Prova 01 - GBD Nome: Matheus José da Costa 11711BCC008

② No gerenciamento de buffers, para discos rígidos, em por exemplo uma operação entre dois arquivos (um para escrita e outro para leitura, que copia o arquivo de um e copia no outro arquivo. Então eu peço 1 byte do 1 arquivo e escrevo no outro. Nesse momento eu carrego um bloco inteiro na memória e daí não preciso me preocupar em carregar o bloco no disco, pois já está na memória RAM. E escritas também serão feitas em um bloco que está na memória RAM. Quanto mais buffers disponíveis no disco, maior será o desempenho da aplicação. E quando eu faço um flush, então estarei persistindo no disco. Para o caso do SSD, não tenho o tempo de SEK - igual acontece com um disco rígido - mas tem um tempo grande para gravação. E em um SSD eu trabalho com blocos, com 1 byte eu leio um bloco inteiro. E diferentemente dos discos rígidos a memória cache (buffers) não é necessário para grande desempenho.

③ Sobre os métodos convencionais para organização de registros de de Tamanho fixo, temos que um dado método requer que os registros tenham um número de bytes pré-definidos; um número de atributos pré-definidos; também podemos iniciar cada registro com um indicador de tamanho consistindo da contagem do número de bytes do registro.

Em um número de bytes pré definidos eu não precisaria armazenar uma quebra de linha. E se eu tentar guardar, por exemplo, um nome de um País de 12 bytes, em um armazenamento de 11 bytes, então não será guardado o último byte.

E usando um separador, a aplicação controla a quantidade e quantidade de registros dentro de uma quantidade de bytes pré-definida, mas daí separa usando um delimitador. E o espaço não usado fica para o final.

data
fecha • •

D S T Q Q S S
D L M M J V S

Na implementação usando tamanho variável no arquivo, temos um arquivo que contém o byte de início de um registro, indexado então os bytes com uma variável de 4 bytes (vetor de inteiros). Para chegar ao valor de um dado registro no arquivo "original", faço a posição que quero buscar, exemplo index 3, daí pego o valor referente a essa posição no arquivo de index (um valor representando onde um registro estará iniciando no arquivo "original") e um valor eu subtraio pelo valor da posição anterior, index 2, e então tenho qual é a posição onde o novo registro inicia e termina (seu comprimento). Nesse método não consegui mos buscar por chave, apenas pela posição e fazendo um seek para a posição inicial, e com o valor obtido anteriormente ainda posso.

Para buscar um registro com o método de tamanho fixo, precisamos acessar sequencialmente, registro por registro, até achar o registro ou chegar no fim do arquivo. Essa busca tem um custo descrito "O(n)".

① O acesso ao disco rígido é um acesso "caro" em comparação com a memória RAM. Quando queremos fazer a movimentação do braço (seeking), temos que é uma tarefa lenta. E para isso temos os cilindros, que se a informação estiver em um cilindro, não precisamos fazer seeking. Mas se tivermos acesso simultâneo a dois arquivos armazenados em cilindros não internos, temos um PIOR caso. Em sistemas multi-usuários é muito custoso, além de que é difícil calcular o tempo de seek, então calcula-se um tempo médio para cada operação. Temos também o atraso rotacional que é o tempo necessário para o disco rotacionar até que o setor desejado esteja embaixo da cabeça de leitura/gravação. Em média um atraso é de meia volta. Isso um alto custo para comportamento aleatório. Temos também o tempo de transferência, que o tempo que leva a leitura de um setor que está sob a cabeça de leitura/gravação. O tempo médio para a leitura de

data
fecha

D S T Q Q S S
D L M M J V S

com arquivo é tempo de seek + atraso rotacional + tempo de transferência. E 50 vetores em trilhas aleatórias porém um custo maior que 50 vetores em trilhas sequenciais, como descrito anteriormente.